



MOBILE APPLICATION DEVELOPMENT

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



AULA 02

Mantendo dados remotos atualizados

Quando reutilizar e quando buscar novamente?



“*Todos os modelos estão errados, mas alguns são úteis.*”
— *George E. P. Box*



VOLTANDO PARA UMA TELA

- Você acessa uma tela que busca dados de uma API.
- Depois navega para outra parte do aplicativo.
- Alguns segundos depois, volta para a tela anterior.

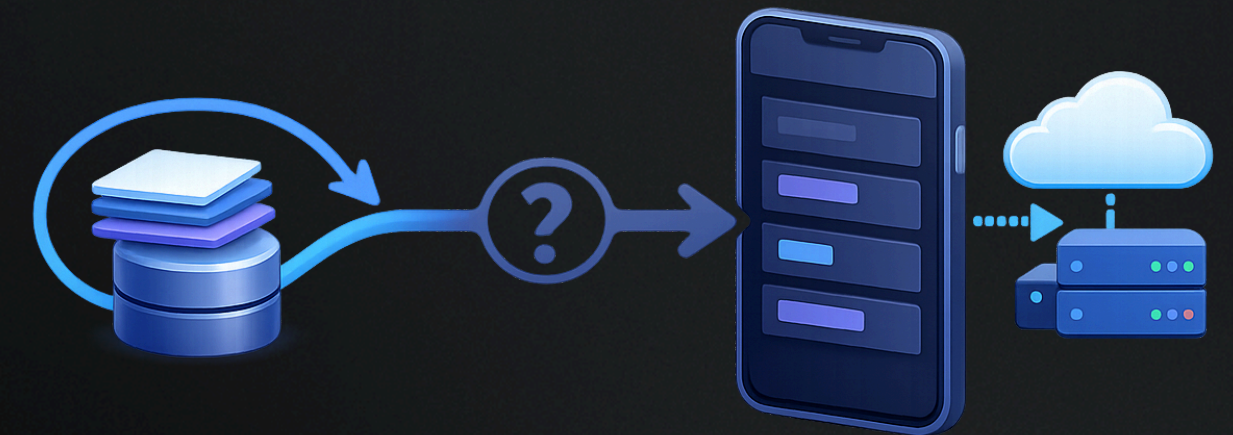


E AGORA?

O aplicativo deveria...

REUTILIZAR OS DADOS QUE JÁ POSSUI?

BUSCAR OS DADOS NOVAMENTE NA API?





Mentimeter

NÃO EXISTE UMA RESPOSTA ÓBVIA

Buscar novamente sempre

Dados mais atuais, mas mais requisições.

Reutilizar sempre

Menos requisições, mas risco de mostrar dados antigos.

QUANDO REUTILIZAR E QUANDO BUSCAR NOVAMENTE?

ENTÃO, COMO O TANSTACK QUERY DECIDE O QUE FAZER?

O COMPORTAMENTO PADRÃO

Ao acessar uma tela, o TanStack Query busca os dados da API.

Quando saímos da tela e voltamos, uma nova requisição pode acontecer.

+ Mesmo que os dados já tenham sido carregados anteriormente.
|

+

OS DADOS NÃO DESAPARECEM

Quando uma consulta termina, o TanStack Query mantém os dados disponíveis.

Isso permite reutilizá-los quando a mesma informação for necessária novamente.

CACHE

DADOS JÁ OBTIDOS PODEM SER MANTIDOS PARA REUTILIZAÇÃO.

O **cache** é onde o TanStack Query mantém os dados obtidos pelas consultas.

Ter um dado no cache não significa que ele ainda seja considerado atual.

QUERYKEY

CADA CONSULTA PRECISA SER IDENTIFICADA.

```
useQuery({  
  queryKey: ["products"],  
  queryFn: getProducts,  
});
```

A `queryKey` identifica os dados armazenados no cache.

Consultas com a mesma chave compartilham os mesmos dados.

FRESH

OS DADOS AINDA SÃO CONSIDERADOS ATUAIS.

Enquanto os dados estão **fresh**, o TanStack Query pode reutilizá-los sem precisar buscar uma versão mais recente.

STALE

OS DADOS CONTINUAM DISPONÍVEIS, MAS PODEM ESTAR DESATUALIZADOS.

Quando os dados ficam **stale**, eles permanecem no cache.

O TanStack Query passa apenas a considerar que pode ser necessário buscar uma versão mais recente.

STALETIME

DEFINE POR QUANTO TEMPO OS DADOS PERMANECEM FRESH.

```
useQuery({  
  queryKey: ["products"],  
  queryFn: getProducts,  
  staleTime: 30_000,  
});
```

Neste exemplo, os dados permanecem **fresh por 30 segundos**.

Depois desse período, tornam-se **stale**.

REFETCH

ATUALIZANDO DADOS QUE JÁ FORAM CARREGADOS

Uma query **stale** pode buscar os dados novamente para obter uma versão mais recente dos dados.

Esse processo é chamado de **refetch**.

O REFETCH ACONTECE QUANDO EXISTE UMA OPORTUNIDADE

Uma query stale pode buscar dados novamente quando, por exemplo:

- um componente passa a utilizar a query;
- a aplicação recupera o foco;
- a conexão com a rede é restabelecida.

VOLTANDO AO NOSSO EXEMPLO

Ao retornar para a tela:

1. os dados ainda estavam no cache;
2. eles estavam `stale`;
3. o componente voltou a utilizar a query;
4. o TanStack Query realizou um `refetch`.

UMA QUERY TAMBÉM POSSUI UM CICLO DE USO

Enquanto algum componente utiliza uma query, ela está **active**.

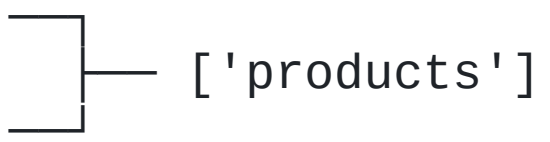
Quando nenhum componente utiliza mais aquela query, ela fica **inactive**.

Os dados continuam no cache.

ACTIVE

EXISTE PELO MENOS UM COMPONENTE UTILIZANDO A QUERY.

Component A
Component B



```
graph LR; A[Component A] --- B[Component B]; A --- C["['products']"]; B --- C;
```

['products']

A query permanece ativa enquanto possuir componentes que utilizam a query .

INACTIVE

NENHUM COMPONENTE ESTÁ UTILIZANDO A QUERY.

A query deixa de estar ativa, mas seus dados não são removidos imediatamente.

Eles continuam disponíveis no cache caso sejam necessários novamente.

GCTIME

DEFINE QUANTO TEMPO UMA QUERY INATIVA PERMANECE NO CACHE.

```
useQuery({  
  queryKey: ["products"],  
  queryFn: getProducts,  
  gcTime: 10_000,  
});
```

Enquanto esse tempo não termina, os dados ainda podem ser reutilizados.

GARBAGE COLLECTION

Quando uma query permanece inativa além do `gcTime`, ela pode ser removida do cache.

Se os dados forem necessários novamente depois disso, uma nova consulta precisará ser realizada.

O CICLO DE UMA QUERY

fresh → **stale**

indica o quanto confiamos na atualidade dos dados.

active → **inactive** → **garbage collected**

indica se a query está sendo utilizada e por quanto tempo será mantida sem consumidores.



DEVTOOLS NO EXPO

O React Query Devtools é uma ferramenta visual para **inspecionar queries, dados e o estado do cache** em tempo real.

```
npx expo install @dev-plugins/react-query
```

```
import { useReactQueryDevTools } from "@dev-plugins/react-query";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

const queryClient = new QueryClient();

export default function App() {
  useReactQueryDevTools(queryClient);

  return (
    <QueryClientProvider client={queryClient}>
      <MyApp />
    </QueryClientProvider>
  );
}
```

Pressione **shift** + **m** no terminal e selecione o plugin **React Query** para abrir sua interface web.

TKDODO'S BLOG

Artigos sobre React, TypeScript e TanStack Query escritos por um dos mantenedores da biblioteca.

Mais do que ensinar a API da ferramenta, os textos discutem como pensar sobre dados remotos e gerenciamento de estado em aplicações reais.

PARA COMEÇAR

Practical React Query

Uma introdução prática aos principais conceitos e decisões por trás do TanStack Query.

 <https://tkdodo.eu/blog/practical-react-query>

LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

  Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2026